

EIP 8282

Smart Contract Security Assessment

July 29, 2026



ABSTRACT

Dedaub was commissioned to perform a security audit of the `sys-asm` repository at commit `83f9801245ff56878a450b5625801101b9a225a1`. This covers the implementation of the builders deposits and exit functionality of the EIP 8282.

BACKGROUND

The deposit and exit request contracts are based on the EIP-7002/EIP-7251 system contracts.

The contracts logic is divided into two main flows based on the caller's address: a "system" path and a "user" path. If the caller is the system address (`0xfffffffffffffffffffffffffffffffffffffe`) the contract runs the system path; otherwise, it runs the user path.

On the user path, if no call data and no value are provided, the contract returns the current request fee. If valid call data and sufficient ETH are sent, the contract adds the user's request to a queue. The required ETH is determined by an exponential function based on the number of excess requests (i.e., those beyond the target). In addition to the fee, submitting builder deposit requests require sending an amount of at least 1 ETH. Each successfully added request increments a counter tracking the total requests for the current block.

On the system path, the contract dequeues up to `MAX_REQUESTS_PER_BLOCK` requests (the maximum is currently set at 64 for deposits and 16 for exits). It should be noted that removing requests from the queue does not delete their used storage slots. After processing these, it resets the request counter and updates the "excess" value, which records how many requests exceeded the target (specified by the `TARGET_REQUESTS_PER_BLOCK` parameter, currently set at 8 for deposits and 2 for exits). Finally, the contract returns the collected requests.

An important difference of the builder request contracts is their newly added ability to pause the contracts: if the system call is performed with non-empty calldata, the excess variable is set to the special EXCESS_INHIBITOR value (equal to uint256.max) disabling the introduction of new requests.

Subsequent system calls with non-empty calldata keep processing the queue.

Finally, a system call with empty calldata will unpause the contract, resetting the excess variable to 0.

SETTING & CAVEATS

This audit report mainly covers the contracts of the `sys-asm` repository at commit `83f9801245ff56878a450b5625801101b9a225a1`, as well as the specification in the EIP (considering the EIP definition at commit `554d3325e31c3f74078402d961355218ece16bee` of the ethereum/EIPs repository).

3 auditors worked on the codebase for 4 days (each) on the following contracts:

src/

builder_deposits/

ctor.eas

main.eas

builder_exits/

ctor.eas

main.eas

common/

fake_expo.eas

bytecode/

builder_deposits/

ctor.hex

main.hex

builder_exits/

ctor.hex
main.hex

Throughout the audit the auditors reviewed the code at numerous levels of abstraction, including the Geas code itself, the disassembled bytecode, as well as the three-address-code (TAC) and decompilation produced by our decompiler. This was done to ensure both human and programmatic understanding of the code, as well as isolating any potential bugs introduced by Geas.

The audit was primarily focused on the security of the smart contracts, and their adherence to the given specification. How the rest of the EIP is implemented into the protocol is outside of the scope of this audit, however, the auditors did attempt to identify potential cross-cutting issues based on the code in scope.

Audit Start Date: July 20, 2026

Report Submission Date: July 24, 2026

The audit's main target is security threats, i.e., what the community understanding would likely call "hacking", rather than the regular use of the protocol. Functional correctness (i.e. issues in "regular use") is a secondary consideration. Typically it can only be covered if we are provided with unambiguous (i.e. full-detail) specifications of what is the expected, correct behavior. In terms of functional correctness, we often trusted the code's calculations and interactions, in the absence of any other specification. Functional correctness relative to low-level calculations (including units, scaling and quantities returned from external protocols) is generally most effectively done through thorough testing rather than human auditing.

CONTRACTS LOGIC OVERVIEW

We next provide the high-level output of our decompiler for the two audited contracts. As the EIP does not provide a detailed and complete description of the functionality of the two contracts, the recovered decompilation output is very useful to anyone wishing to understand them.

Alternatively, one can look at the Three-Address Code (TAC) recovered by our lifter (Gigahorse): <https://gist.github.com/sifislaq/1358e4078239756e6dd96325bbde8605>. This representation, having abstracted away the stack to introduce registers, sits in the middle between the Geas source/disassembly and the high-level decompilation.

Deposit Contract

```
contract DepositQueue {
    // Variables inferred from the use of storage instructions
    uint256 _excessWithdrawals; // STORAGE[0x0]
    uint256 _withdrawalReqCount; // STORAGE[0x1]
    uint256 _queueStart; // STORAGE[0x2]
    uint256 _queueEnd; // STORAGE[0x3]

    function main(uint256 calldata_0_32, uint256 calldata_32_64, uint64
calldata_80_88, uint256 calldata_64_96, uint256 calldata_96_128, uint256
calldata_128_160, uint256 calldata_160_192) public payable {
        if (0xffffffffffffffffffffffffffffffffffffffff == msg.sender) {
            v0 = v1 = _queueEnd - _queueStart;
            if (64 <= v1) {
                v0 = v2 = 64;
            }
            v3 = v4 = 0;
            while (v3 != v0) {
                MEM[184 * v3] = STORAGE[4 + 6 * (v3 + _queueStart)];
                MEM[32 + 184 * v3] = STORAGE[5 + 6 * (v3 + _queueStart)];
                MEM[64 + 184 * v3] = STORAGE[6 + 6 * (v3 + _queueStart)];
                v5 = uint64(STORAGE[6 + 6 * (v3 + _queueStart)] >> 64);
                MEM8[87 + 184 * v3] = v5 >> 56 & 0xFF;
                MEM8[86 + 184 * v3] = v5 >> 48 & 0xFF;
                MEM8[85 + 184 * v3] = v5 >> 40 & 0xFF;
            }
        }
    }
}
```

```

MEM8[84 + 184 * v3] = v5 >> 32 & 0xFF;
MEM8[83 + 184 * v3] = v5 >> 24 & 0xFF;
MEM8[82 + 184 * v3] = v5 >> 16 & 0xFF;
MEM8[81 + 184 * v3] = v5 >> 8 & 0xFF;
MEM8[80 + 184 * v3] = v5 & 0xFF;
MEM[96 + 184 * v3] = STORAGE[7 + 6 * (v3 + _queueStart)];
MEM[128 + 184 * v3] = STORAGE[8 + 6 * (v3 + _queueStart)];
MEM[160 + 184 * v3] = STORAGE[9 + 6 * (v3 + _queueStart)];
v3 += 1;
}
if (_queueEnd == _queueStart + v0) {
    _queueStart = 0;
    _queueEnd = 0;
} else {
    _queueStart = _queueStart + v0;
}
}
if (msg.data.length) {
    v6 = v7 = uint256.max;
} else {
    if (uint256.max != _excessWithdrawals) {
        if (_withdrawalReqCount + _excessWithdrawals > 8) {
            v6 = v8 = _withdrawalReqCount + _excessWithdrawals - 8;
        }
        else {
            v6 = v9 = 0;
        }
    }
    else {
        v6 = v9 = 0;
    }
}
_excessWithdrawals = v6;
_withdrawalReqCount = 0;
return MEM[0:184 * v3];
} else {
    v10 = v11 = _excessWithdrawals;
    require(uint256.max != v11);
    if (_withdrawalReqCount > 8) {
        v10 = v12 = _withdrawalReqCount - 8 + v11;
    }
    v13 = v14 = 17;
    v15 = v16 = 1;
    v17 = v18 = 0;
    while (v13 > 0) {
        v17 += v13;
    }
}

```

```

        v13 = v10 * v13 / (v15 * 17);
        v15 += 1;
    }
    if (184 == msg.data.length) {
        require(msg.value >= v17 / 17);
        require(10 ** 9 <= calldata_80_88);
        require(msg.value - v17 / 17 >= 10 ** 9 * calldata_80_88);
        _withdrawalReqCount += 1;
        STORAGE[4 + 6 * _queueEnd] = calldata_0_32;
        STORAGE[5 + 6 * _queueEnd] = calldata_32_64;
        STORAGE[6 + 6 * _queueEnd] = calldata_64_96;
        STORAGE[7 + 6 * _queueEnd] = calldata_96_128;
        STORAGE[8 + 6 * _queueEnd] = calldata_128_160;
        STORAGE[9 + 6 * _queueEnd] = calldata_160_192; // last 8 bytes are empty
        // CALLDATACOPY(0, 0, 184);
        log calldata[0:184];
        _queueEnd += 1;
        exit;
    } else {
        require(!msg.data.length);
        require(!msg.value);
        return v17 / 17;
    }
}
}
}
}

```

Exit Contract

```

contract ExitQueue {
    // Variables inferred from the use of storage instructions
    uint256 _excessWithdrawals; // STORAGE[0x0]
    uint256 _withdrawalReqCount; // STORAGE[0x1]
    uint256 _queueStart; // STORAGE[0x2]
    uint256 _queueEnd; // STORAGE[0x3]

    function main(uint256 calldata_0_32, uint256 calldata_32_64) public payable {
        if (0xffffffffffffffffffffffffffffffff == msg.sender) {
            v0 = v1 = _queueEnd - _queueStart;
            if (16 <= v1) {
                v0 = v2 = 16;
            }
        }
    }
}

```

```

}
v3 = v4 = 0;
while (v3 != v0) {
    MEM[68 * v3] = STORAGE[4 + 3 * (v3 + _queueStart)] << 96;
    MEM[20 + 68 * v3] = STORAGE[5 + 3 * (v3 + _queueStart)];
    MEM[52 + 68 * v3] = STORAGE[6 + 3 * (v3 + _queueStart)];
    v3 += 1;
}
if (_queueEnd == _queueStart + v0) {
    _queueStart = 0;
    _queueEnd = 0;
} else {
    _queueStart = _queueStart + v0;
}
if (msg.data.length) {
    v5 = v6 = uint256.max;
} else {
    if (uint256.max != _excessWithdrawals) {
        if (_withdrawalReqCount + _excessWithdrawals > 2) {
            v5 = v7 = _withdrawalReqCount + _excessWithdrawals - 2;
        }
        else {
            v5 = v8 = 0;
        }
    }
    else{
        v5 = v8 = 0;
    }
}
_excessWithdrawals = v5;
_withdrawalReqCount = 0;
return MEM[0:68 * v3];
} else {
    v9 = v10 = _excessWithdrawals;
    require(uint256.max != v10);
    if (_withdrawalReqCount > 2) {
        v9 = v11 = _withdrawalReqCount - 2 + v10;
    }
    v12 = v13 = 17;
    v14 = v15 = 1;
    v16 = v17 = 0;
    while (v12 > 0) {
        v16 += v12;
        v12 = v9 * v12 / (v14 * 17);
        v14 += 1;
    }
}

```

```

    }
    if (48 == msg.data.length) {
        require(msg.value >= v16 / 17);
        _withdrawalReqCount += 1;
        STORAGE[4 + 3 * _queueEnd] = msg.sender;
        STORAGE[5 + 3 * _queueEnd] = calldata_0_32;
        STORAGE[6 + 3 * _queueEnd] = calldata_32_64; // last 16 bytes are empty
        // MEM[0] = msg.sender << 96;
        // CALLDATACOPY(20, 0, 48);
        log msg.sender ++ calldata[0:48];
        _queueEnd += 1;
        exit;
    } else {
        require(!msg.data.length);
        require(!msg.value);
        return v16 / 17;
    }
}
}
}
}

```

PROTOCOL-LEVEL CONSIDERATIONS

ID	Description	STATUS
P1	Geas assembly introduces avoidable implementation complexity and risk	INFO
The contracts are written directly in Geas, a minimal assembler with no structured control-flow abstractions. As a result, correctness must be reasoned about manually at the level of raw opcodes and stack manipulation (or using lifting tools), rather than expressed and checked at a higher level, and verified by auditors the same way. This also makes the contracts harder to develop and maintain in the first place: every change requires the author to manually track stack depth and control flow by hand,		

raising the bar for contributors and increasing the chance that a routine edit introduces a subtle error. We recommend Yul instead: still low-level and gas-predictable, but with basic structured constructs (named variables, function scoping, control flow), and compiled through `solc`: an established, widely audited toolchain with far more tooling and auditor familiarity. This wouldn't change the audited design, but it would lower the odds and cost of implementation-level bugs going forward, and make future revisions easier to write and review.

VULNERABILITIES & FUNCTIONAL ISSUES

This section details issues affecting the functionality of the contract. Dedaub generally categorizes issues according to the following severities, but may also take other considerations into account such as impact or difficulty in exploitation:

Category	Description
CRITICAL	Can be profitably exploited by any knowledgeable third-party attacker to drain a portion of the system's or users' funds OR the contract does not function as intended and severe loss of funds may result.
HIGH	Third-party attackers or faulty functionality may block the system or cause the system or users to lose funds. Important system invariants can be violated.
MEDIUM	Examples: • User or system funds can be lost when third-party systems misbehave. • DoS, under specific conditions. • Part of the functionality becomes unusable due to a programming error.
LOW	Examples: • Breaking important system invariants but without apparent consequences. • Buggy functionality for trusted users where a workaround exists. • Security issues which may manifest when the system evolves.

Issue resolution includes “dismissed” or “acknowledged” but no action taken, by the client, or “resolved”, per the auditors.

CRITICAL SEVERITY:

[No critical severity issues]

HIGH SEVERITY:

[No High severity issues]

MEDIUM SEVERITY:

[No medium severity issues]

LOW SEVERITY:

[No Low severity issues]

OTHER / ADVISORY ISSUES:

This section details issues that are not thought to directly affect the functionality of the project, but we recommend considering them.

ID	Description	STATUS
A1	Pause/unpause functionality is not described in the EIP	INFO
The EIP specifies the system-call path only as a dequeue-and-update-excess operation; EXCESS_INHIBITOR is mentioned solely as a deployment-time value cleared by the first system call, with no text describing the repeatable pause/unpause mechanism supported by the contracts. Since triggering this pause/unpause requires altering the way the system call is performed, exercising it requires a hard fork. As a		

result, the authors should specify how and when pausing is meant to be triggered so that intent is settled ahead of time rather than left to be inferred from bytecode.

A2

Contract pausing functionality can break system invariants

OPEN

If a request contract is paused by setting its excess value to the special `EXCESS_INHIBITOR` value, and resumed, its excess value is reset to 0, regardless of whether its queue has been fully processed or not.

As a result, any pending requests are not accounted for in the value of the excess variable and following requests have a smaller fee than they should.

We suggest the excess value is approximated to a value of `max(tail_idx - head_idx - TARGET_REQUESTS_PER_BLOCK, 0)`.

A3

Deposit registrations are open before the fork

INFO

Unlike the exits contract (and the deployed EIP-7002/EIP-7251 contracts), the deposit contract's constructor does not write `EXCESS_INHIBITOR`, so the contract accepts paid registrations from the moment it is deployed, before the fork activates – the team has confirmed this is intentional. The following two consequences are not necessarily problematic, but they are worth documenting. First, the contract is a pure accounting sink with no refund path: ETH sent pre-fork can only be credited via CL processing after activation, so a fork delay locks the funds (at least 1 ETH plus fee per request) indefinitely, and a fork abort – or a redesign requiring redeployment – loses them permanently. Second, since no system calls occur pre-fork, the per-block request counter is never reset: the fee grows monotonically with the cumulative number of pre-fork registrations, and the first post-fork system call converts the entire accumulated count into excess, producing a fee spike at activation proportional to the pre-fork backlog.

DISCLAIMER

The audited contracts have been analyzed using automated techniques and extensive human inspection in accordance with state-of-the-art practices as of the date of this report. The audit makes no statements or warranties on the security of the code. On its own, it cannot be considered a sufficient assessment of the correctness of the contract. While we have conducted an analysis to the best of our ability, it is our recommendation for high-value contracts to commission several independent audits, a public bug bounty program, as well as continuous security auditing and monitoring through Dedaub Security Suite.

ABOUT DEDAUB

Dedaub offers significant security expertise combined with cutting-edge program analysis technology to secure some of the most prominent protocols in DeFi. The founders, as well as many of Dedaub's auditors, have a strong academic research background together with a real-world hacker mentality to secure code. Protocol blockchain developers hire us for our foundational analysis tools and deep expertise in program analysis, reverse engineering, DeFi exploits, cryptography and financial mathematics.